第七章：其他开源空间数据处理库

谭振宇

西北大学城市与环境学院

2026 年 04 月 20 日

其他开源库的使用

Fiona 矢量数据处理

Rasterio 栅格数据处理

GeoPandas 矢量数据处理

空间多维数据处理工具包

CartoPy 地图绘制

课后练习

学习目标

西北大学

核心学习目标

- 使用 Fiona 库进行矢量数据的读写
- 使用 Rasterio 库进行栅格数据的读写
- 使用 GeoPandas 库进行矢量数据读写
- 使用 NetCDF4、PyHDF、H5Py 库进行多维科学数据的读取
- 使用 XArray 库进行多维数据的读写和操作
- 使用 CartoPy 进行简单地图绘制

背景介绍

GDAL 是使用 C++ 语言编写的一个用于栅格和矢量地理空间数据库，在使用 Python 进行调用时，难免会出现语言风格之间的不协调。所以 Python 社区开发了其他**更加 Pythonic** 的空间数据处理库：

Fiona & Rasterio

- **Fiona** 专用于空间矢量数据的处理
- **Rasterio** 专用于栅格数据的处理

科学数据与制图

- 专门用于读写 NetCDF 和 HDF 格式的多维科学数据专有库，如 **NetCDF4**、**PyHDF**、**H5Py**、**XArray**
- **CartoPy** 用于简单地图的绘制

其他开源库的使用

Fiona 矢量数据处理

Rasterio 栅格数据处理

GeoPandas 矢量数据处理

空间多维数据处理工具包

CartoPy 地图绘制

课后练习

Fiona 简介

`Fiona` 基于 GDAL 中的 OGR 部分进行二次包装，以 Python 原生的数据结构和语言规范对外提供 API，更加方便 Python 开发者对空间矢量数据进行操作。

基本上，用 GDAL 的 Python 绑定 API 编写程序时给人一种仍然在写 C/C++ 的感觉，而 Fiona 基于 GDAL 提供了更加 Pythonic 的读取 API。

数据结构对比

- **GDAL/OGR**：数据源 (DataSource) -> 图层 (Layer) -> 要素 (Feature) -> 属性和几何体 (Attributes and Geometry)
- **Fiona**：采用 Python 内置的数据结构。一个要素以 GeoJSON 表示，使用字典 (`dict`) 组织；一个图层包含在一个集合 (`Collection`) 中，可以进行迭代遍历。

使用 Fiona 读取矢量数据

使用 Fiona 操作空间数据就像操作 Python 内置的数据结构一样简单：

```python
import fiona
from fiona import crs

fp = r"C:\Users\tanzhenyu\Dataware\GeoPy\XiAn\XiAn.shp"
with fiona.open(fp) as fc:
    # fc是一个Collection对象
    print(f'文件采用{crs.to_string(fc.crs)}参考系统')
    print(f'要素的地理范围为{fc.bounds}')
    print(f'文件中包含{len(fc)}个要素')

    # fc中的每个元素都是一个要素（dict），包含geometry和properties
    print(f"文件中第一个要素类型为{fc[0]['geometry']['type']}")
    print(f"文件中第一个要素包含的属性{fc[0]['properties']}")

    # 遍历读取
    # for g in fc:
    #     print(g['properties']['NAME'])
```

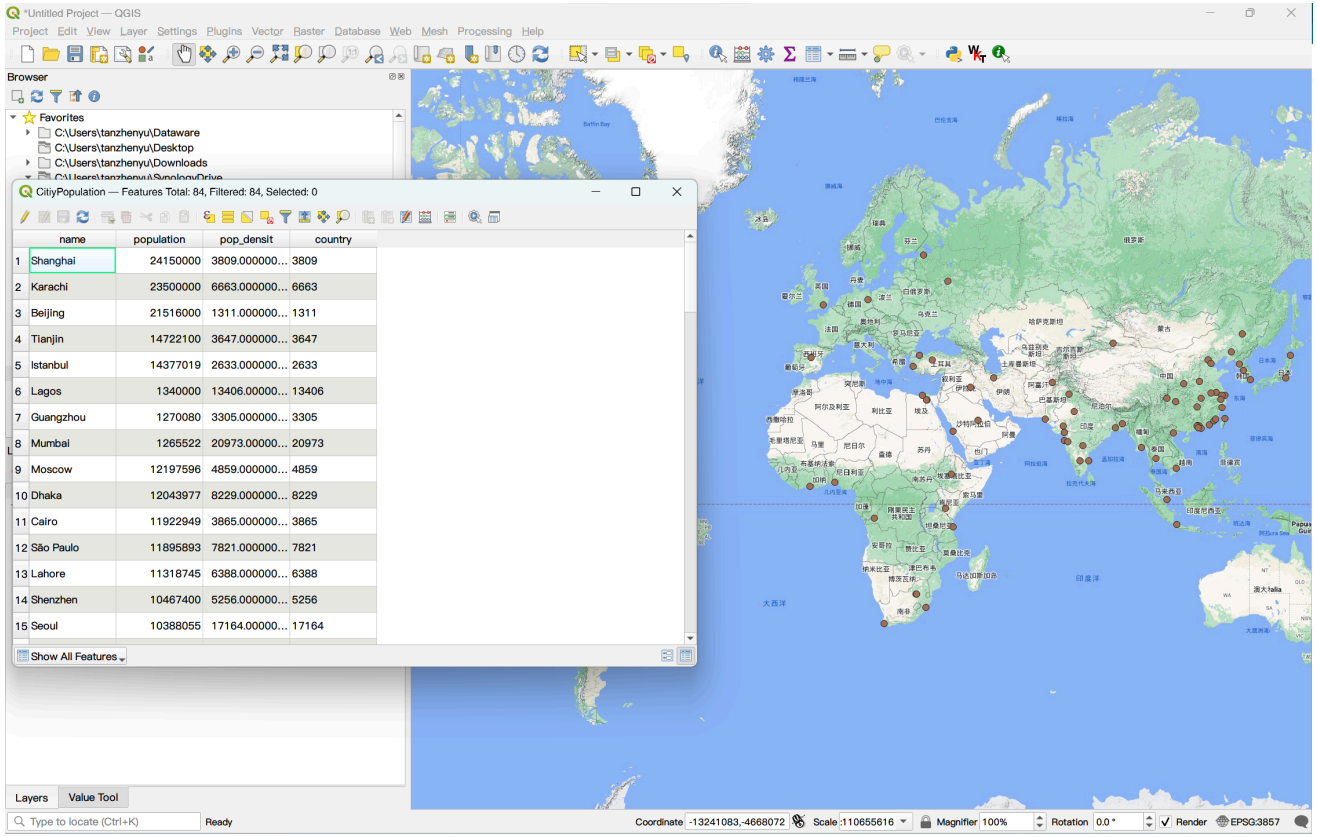
基于 Fiona 创建矢量数据

西北大学

可以通过定义结构模式 (Schema)，将带有地理坐标的表格数据转化为空间要素。

```python
import pandas
import fiona
from shapely.geometry import Point, mapping

schema = {
    'geometry': 'Point',
    'properties': {'name': 'str', 'population': 'int'}
}
def row_to_shape(row, dst):
    point = Point(float(row['longitude']), float(row['latitude']))
    prop = {'name': row['name'], 'population': int(row['population'])}
    dst.write({'geometry': mapping(point), 'properties': prop})

with fiona.open('CitiyPopulation.shp', 'w', crs=fiona.crs.from_epsg(4326),
                driver='ESRI Shapefile', schema=schema) as dst:
    data = pandas.read_csv('CitiyPopulation.csv', delimiter=',')
    data.apply(lambda row: row_to_shape(row, dst), axis=1)
```

基于 Fiona 创建矢量地图示例



使用 Fiona 将 CSV 文本数据转换为 Shapefile 矢量数据并在 QGIS 中可视化

Fiona 小结

西北大学

读取数据

使用 fiona.open() 函数配合 Python with 语句打开文件，可以得到一个空间要素的集合 (Collection)，通过遍历该集合可以获取每个空间实体要素的信息（结构为 dict）。

写入数据

使用 fiona.open() 函数读模式打开文件，必须通过传递一个预定义好的 schema（字典）进行数据写入。数据写入时，可以通过 for 循环语句依次写入，或者基于 DataFrame 使用 apply() 函数进行处理。

其他开源库的使用

Fiona 矢量数据处理

Rasterio 栅格数据处理

GeoPandas 矢量数据处理

空间多维数据处理工具包

CartoPy 地图绘制

课后练习

Rasterio 简介

Rasterio 是 MapBox 旗下的开源库，是基于 GDAL 库二次封装的更加符合 Python 风格的空间栅格数据处理模块。它是对 GDAL 库中栅格数据读写等简单功能的替代。

地理仿射变换

在 Rasterio 1.0 以后，对于 GeoTransform 的表示弃用了 GDAL 风格的仿射变换，使用了 `affine` 库风格。

旧（GDAL）：`(c, a, b, f, d, e)`

新（Rasterio）：`affine.Affine(a, b, c, d, e, f)`

计算坐标时，直接使用行列号与仿射变换对象相乘即可，符合矩阵乘法操作，更加直观。

使用 Rasterio 读取遥感影像数据

西北大学

```python
import rasterio

fn = r"C:\Users\tanzhenyu\Dataware\GeoPy\DataForBook\XiAn-202108.tif"
with rasterio.open(fn) as ds:
    print(f'数据格式: {ds.driver}, 波段数目: {ds.count}')
    print(f'影像尺寸: {ds.width} x {ds.height}')
    print(f'地理范围: {ds.bounds}')
    print(f'投影定义: {ds.crs}')
    print(f'仿射变换参数: \n{ds.transform}')

    # 获取第一个波段数据，跟GDAL一样索引从1开始
    band1 = ds.read(1)
    print(f'第一波段的极值: {band1.max()} / {band1.min()}')

    # 坐标转换示例
    x, y = (ds.bounds.left + 300, ds.bounds.top - 300)
    row, col = ds.index(x, y) # 根据地理坐标得到行列号
    x, y = ds.xy(row, col) # 根据行列号得到中心点地理坐标
    x, y =  ds.transform * (row, col) # 根据行列号得到左上角地理坐标
```

基于 Rasterio 进行栅格格式和投影转换

西北大学

```python
import numpy as np
import rasterio
from rasterio.warp import calculate_default_transform, reproject, Resampling

src_img, dst_img = "XiAn-202108-AOI.tif", "XiAn-202108-AOI.img"
dst_crs = rasterio.crs.CRS.from_epsg('4326')

with rasterio.open(src_img) as src_ds:
    profile = src_ds.profile
    # 计算在新空间参考系下的仿射变换参数、图像尺寸
    dst_transform, dst_width, dst_height = calculate_default_transform(
        src_ds.crs, dst_crs, src_ds.width, src_ds.height, *src_ds.bounds)
    # 更新数据集的元数据信息
    profile.update({'driver': 'HFA', 'crs': dst_crs, 'transform': dst_transform,
                    'width': dst_width, 'height': dst_height, 'nodata': 0})

    # 重投影并分波段写入数据
    with rasterio.open(dst_img, 'w', **profile) as dst_ds:
        for i in range(1, src_ds.count + 1):
            src_array = src_ds.read(i)
            dst_array = np.empty((dst_height, dst_width), dtype=profile['dtype'])
            reproject(source=src_array, src_crs=src_ds.crs, src_transform=src_ds.transform,
                      destination=dst_array, dst_transform=dst_transform, dst_crs=dst_crs,
                      resampling=Resampling.cubic, num_threads=2)
            dst_ds.write(dst_array, i)
```

Rasterio 小结

西北大学

文件读取与写入

使用 `rasterio.open()` 函数，可以通过 Dataset 的 `read(i)` 读取第 i 个波段为 NumPy `ndarray`。
写入时通过传递包含元数据（如文件类型、宽、高、仿射变换、投影）的 `profile` 字典，并用 `write(array, i)` 写入指定波段。

重投影与参数计算

当数据尺寸和空间参考发生变化时（如投影转换重采样），其仿射变换参数必然发生变化。Rasterio 中通过 `calculate_default_transform()` 计算新的参数和尺寸，结合 `reproject()` 实现数据重投影。

其他开源库的使用

Fiona 矢量数据处理

Rasterio 栅格数据处理

GeoPandas 矢量数据处理

空间多维数据处理工具包

CartoPy 地图绘制

课后练习

GeoPandas 简介

GeoPandas 是一个基于 Pandas 的开源 Python 库，专门用于地理空间数据的处理和分析。它结合了 Shapely、Fiona 等库的功能，并提供了两种主要的数据结构：

GeoSeries

扩展了 Pandas 的 Series，用于存储地理空间几何对象。是只有一列的 GeoDataFrame，该列的类型是几何体类型 (Geometry)。

GeoDataFrame

扩展了 Pandas 的 DataFrame，其中有一列是 Geometry 类型（要素的几何信息），其他列是非 Geometry 类型（要素的属性信息）。

基于 GeoPandas 读取空间矢量数据

西北大学

```python
import geopandas as gpd

fn = gpd.datasets.get_path("naturalearth_lowres")
world = gpd.read_file(fn)
print(world.head())
```

```
      pop_est      continent                     name iso_a3  gdp_md_est  \
0    889953.0        Oceania                     Fiji    FJI        5496
1  58005463.0         Africa                 Tanzania    TZA       63177
                                            geometry
0  MULTIPOLYGON (((180.00000 -16.06713, 180.00000...
1  POLYGON ((33.90371 -0.95000, 34.07262 -1.05982...
```

仅仅通过 `gpd.read_file()` 即可非常方便地加载不同格式的空间矢量数据。

基于 GeoPandas 格式转换

西北大学

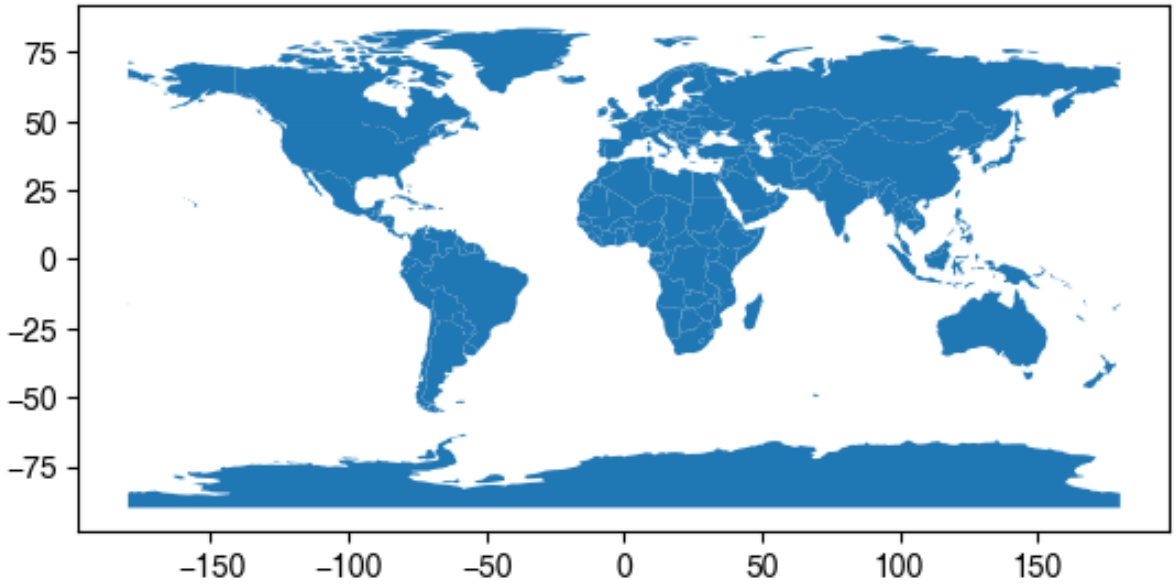
将非空间表格数据（CSV 等）转换为带几何列的 Shapefile：

```python
import pandas as pd
import geopandas as gpd

df = pd.read_csv("CitiyPopulation.csv", delimiter=',')
# 转换为字符串
df[['name', 'country']] = df[['name', 'country']].apply(str)

# 从经纬度列构建几何点
gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(df.longitude, df.latitude)
)

# 写入文件
gdf.to_file('CitiyPopulation.shp')
```

GeoPandas 高级应用与可视化

GeoPandas 与 Matplotlib 结合，可以轻松绘制地图，仅需通过 .plot() 函数：

```python
import geopandas as gpd

world = gpd.read_file(gpd.datasets.get_path("naturalearth_lowres"))
world.plot()
```

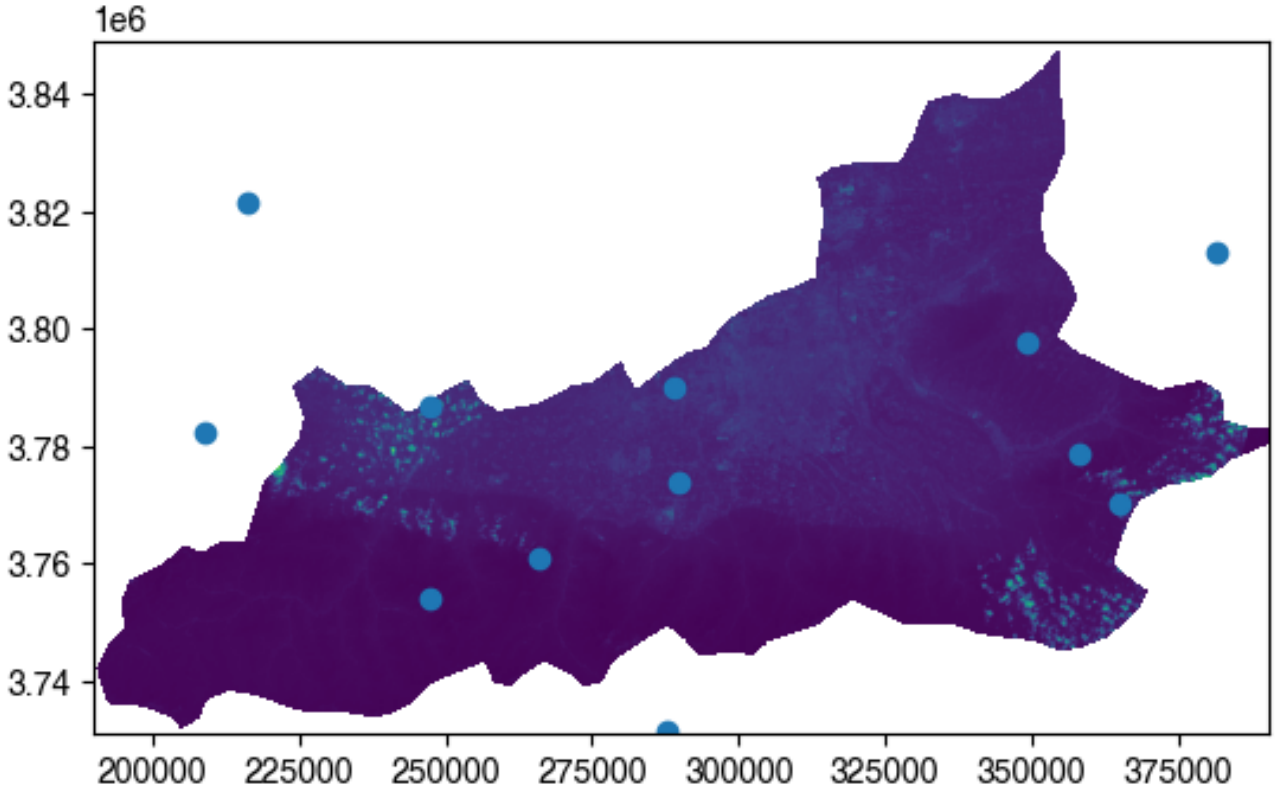


GeoPandas 与 Rasterio 结合应用

一个非常典型的应用场景：在遥感影像上叠加快量点，提取点对应位置的像素值作为样本。

```python
import geopandas as gpd
import rasterio
from rasterio.plot import show
import matplotlib.pyplot as plt

with rasterio.open("XiAn-202108-AOI.tif") as src:
    # 1. 创建随机的样点 GeoDataFrame
    # ... (生成 random 点并包装为 gdf) ...

    # 2. 联合绘图
    fig, ax = plt.subplots()
    ax = rasterio.plot.show(src, ax=ax) # 画底图栅格
    gdf.plot(ax=ax)                     # 画矢量点

    # 3. 提取对应坐标像元值
    coord_list = [(x,y) for x,y in zip(gdf['geometry'].x, gdf['geometry'].y)]
    values = [x for x in src.sample(coord_list)] # 使用 src.sample 采样

    # 4. 拼接至原始 Pandas
    # ...
```

地图叠加与样点提取结果



基于 GeoPandas 和 Rasterio 的联合空间信息可视化

其他开源库的使用

Fiona 矢量数据处理

Rasterio 栅格数据处理

GeoPandas 矢量数据处理

空间多维数据处理工具包

CartoPy 地图绘制

课后练习

为什么需要专有库?

多维数组类型的数据集（如 NetCDF、HDF 等）常用于长时间序列的科学数据集。一个数据文件中可能包含许多数据子集。

基于 GDAL/Rasterio

可以像提取 GeoTIFF 波段那样读取：先获取子数据集（`GetSubDatasets()` 或 `.subdatasets`），然后视为独立栅格再次打开。

基于专门多维库

使用 `netCDF4`、`pyhdf`、`h5py` 或更高阶的 `XArray`。它们能完美识别多维数据的变量、维度和坐标信息，极大简化高维分析工作。

基于 GDAL/Rasterio 读取 HDF

西北大学

```python
# 使用 GDAL
from osgeo import gdal
ds_in = gdal.Open('example.hdf')
subdatasets = ds_in.GetSubDatasets() # 取出所有子数据集路径
band1 = gdal.Open(subdatasets[0][0])  # 取出第1个子数据集（如反射率产品）
arr_b1 = band1.ReadAsArray()

# 使用 Rasterio
import rasterio
with rasterio.open('example.hdf') as ds:
    for name in ds.subdatasets:
        pass
    with rasterio.open(ds.subdatasets[0]) as ds_sub: # 读取第一个波段
        data = ds_sub.read()
```

使用专有库读取 (NetCDF4 / PyHDF)

西北大学

```python
# 使用 netCDF4
from netCDF4 import Dataset
with Dataset('example.nc') as ds:
    print(ds.variables) # 查看内部变量
    data = ds.variables['LU_INDEX'][:] # 提取变量为 MaskedArray
    print(data.shape)

# 使用 pyhdf (针对 HDF4)
from pyhdf.SD import SD, SDC
ds = SD('example.hdf', SDC.READ)
subdatasets = ds.datasets()
ds_bnd = ds.select(list(subdatasets.keys())[0])
data = ds_bnd.get() # 将 HDF4子数据集转换为 NumPy ndarray
ds.end()

# 使用 h5py (针对 HDF5)
import h5py
with h5py.File('example.h5', 'r') as ds:
    band = ds['bands/Rw555']
    data = band[:]
```

基于 XArray 库处理科学多维数据

XArray 是建立在 NumPy 和 Pandas 上，带有标签的多维数组处理库。它不仅能处理数组，还能保留经度、纬度、时间等元数据。

XArray 核心结构

- **DataArray**：基本数据结构，附带标签和维度信息。
- **Dataset**：一个容器，可以包含多个 DataArray，对应多变量整体文件。包含了 Variable（变量：降水、温度等），Dimension（维度：时间、空间等），Coordinate（坐标系：地理和时间刻度）。

使用 XArray 多维切片

借用类似 Pandas 的 .sel()（按标签名称过滤）和 .isel()（按索引过滤），可以无缝查询不同维度的时间-空间面板。

```python
import xarray as xr
from datetime import datetime

# 打开 netcdf，直接变成一个 Dataset
with xr.open_dataset('example.nc') as ds:
    print(ds.data_vars) # 变量
    print(ds.coords)    # 坐标系

    data = ds['tp']     # 获得一个 DataArray

    # 按索引切片
    array = data.isel(hybas_id=0)

    # 按时间标签进行查询切片
    array = data.sel(time=datetime(2000, 1, 1),
                     hybas_id=slice('2120313970', '2120320920'))
```

其他开源库的使用

Fiona 矢量数据处理

Rasterio 栅格数据处理

GeoPandas 矢量数据处理

空间多维数据处理工具包

CartoPy 地图绘制

课后练习

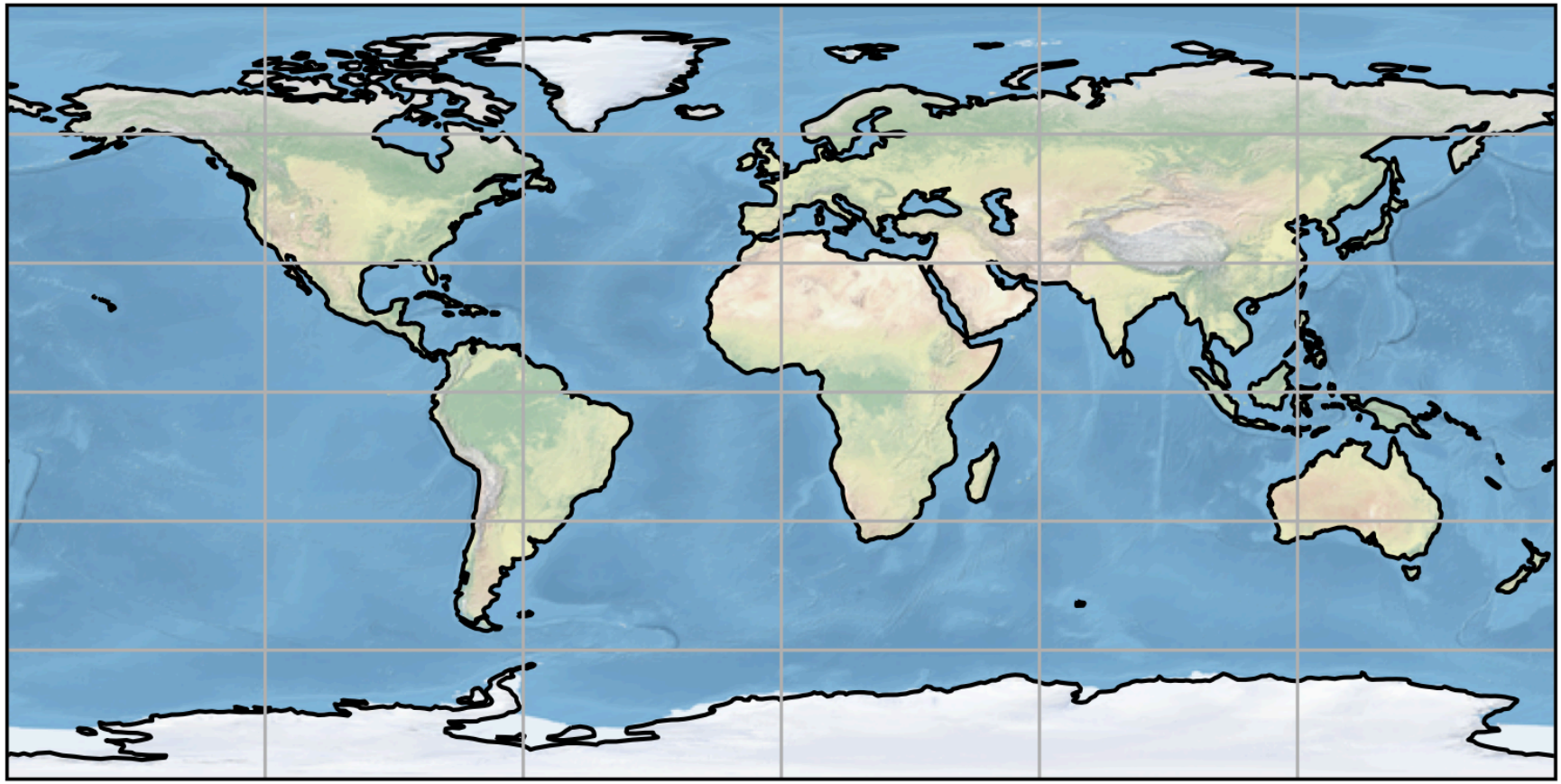
CartoPy 简介

CartoPy 是一个构建于 Matplotlib 之上的 Python 绘图库，内置多种投影支持（如 ccrs.PlateCarree()、ccrs.Mercator() 等）。

利用 CartoPy，可以为 Matplotlib 的 Axes 指定地理投影，然后优雅地添加海岸线、国界、河流等地学图层。

```python
import matplotlib.pyplot as plt
import cartopy.crs as ccrs

plt.figure()
# 定义等距圆柱投影，对 WGS84 制图常用
ax = plt.axes(projection=ccrs.PlateCarree())

ax.stock_img()    # 添加低分辨率地形图背景
ax.coastlines()   # 添加海岸线
ax.gridlines()    # 添加格网
plt.show()
```

CartoPy 绘制世界地图效果展示

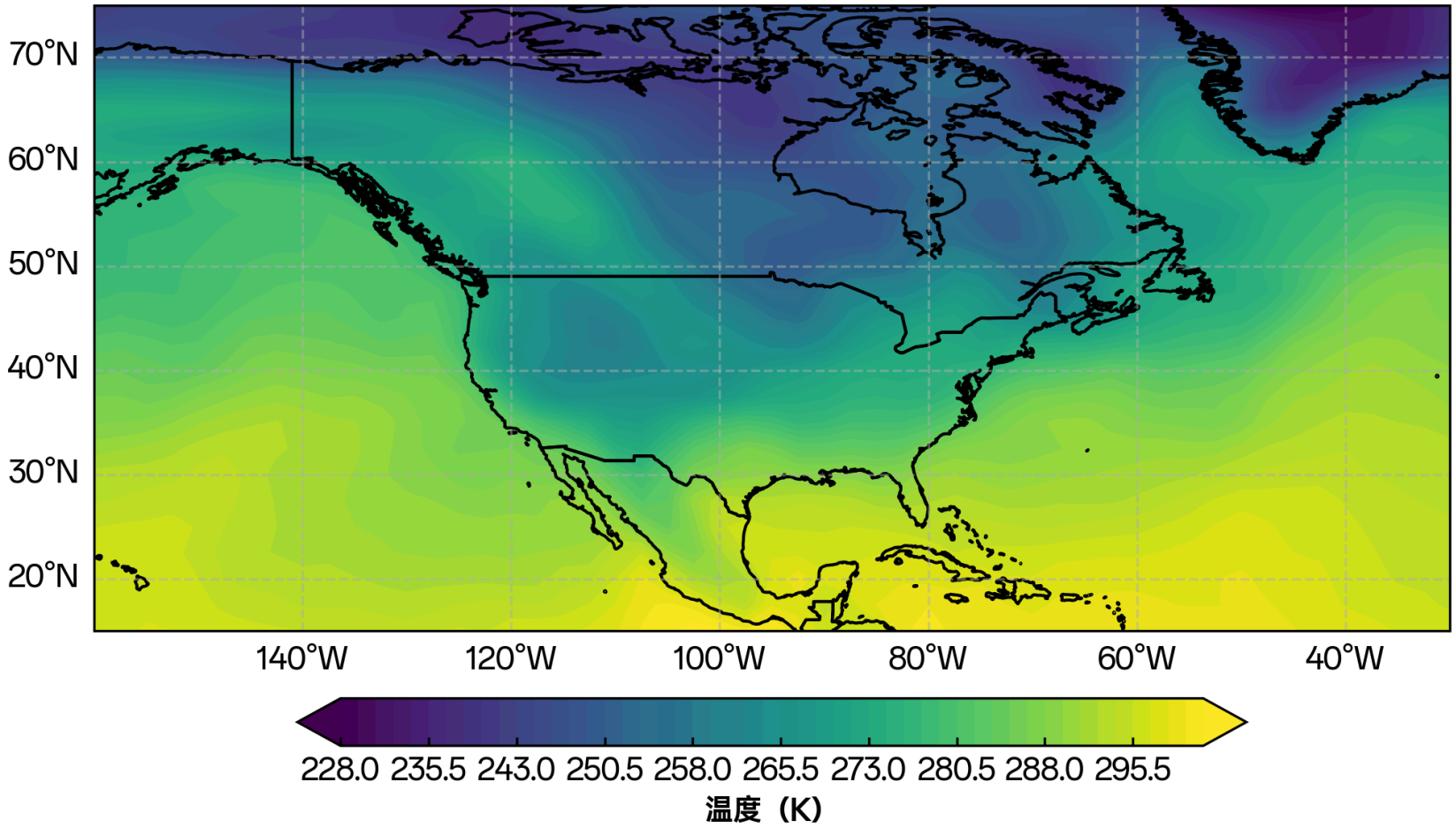


结合 CartoPy 绘制矢量和栅格数据

西北大学

```python
import cartopy.crs as ccrs
import matplotlib.pyplot as plt
import numpy as np
import netCDF4 as nc
import cartopy.io.shapereader as shpreader

# 读取 NC 的温度变量
ds = nc.Dataset('air_temperature.nc')
lon, lat, temp = ds.variables['lon'][:], ds.variables['lat'][:], ds.variables['air'][0]
# 读取 Shapefile
boundary = shpreader.Reader('world-boundaries.shp').geometries()

proj = ccrs.PlateCarree()
fig = plt.figure(figsize=(8, 5))
ax = plt.axes(projection=proj)

# 1. 绘制等值面填充图（栅格）
lon_grid, lat_grid = np.meshgrid(lon, lat)
cf = ax.contourf(lon_grid, lat_grid, temp, levels=50, transform=proj)
fig.colorbar(cf, shrink=0.70, orientation='horizontal')

# 2. 绘制矢量边界（矢量）
ax.add_geometries(boundary, proj, facecolor='none', edgecolor='k')

# 3. 添加网格经纬线
ax.gridlines(draw_labels=True, linestyle='--')
plt.show()
```

CartoPy 联合制图效果展示



其他开源库的使用

Fiona 矢量数据处理

Rasterio 栅格数据处理

GeoPandas 矢量数据处理

空间多维数据处理工具包

CartoPy 地图绘制

课后练习

本章小结

本章涉及内容繁杂，库的底层仍然涉及到 GDAL 库中对于矢量数据和栅格数据模型的抽象表示，理解这些是学好其他第三方空间数据处理库的前提。主要内容包括：

- 基于 **Fiona** 的矢量数据读写：需要体会其数据模型结构的便捷。
- 基于 **Rasterio** 的栅格数据读写：重点掌握仿射变换及其数据传递使用。
- 基于 **GeoPandas** 的矢量与表格融合操作方法。
- 能够使用不同专有库 (**NetCDF4**、**PyHDF**) 以及 **XArray** 高级库处理多维科学数据。
- 能够使用 Matplotlib 配合 **CartoPy** 进行丰富的地理地图绘制。

牛刀小试

作业 1：鄱阳湖水体面积提取与对比

下载我国最大的淡水湖（鄱阳湖）在十年前和今年相同月份的 Landsat 遥感图像。
基于 NDWI（Normalized Difference Water Index）指数提取在这两个年份的湖泊面积，并计算面积的变化率。

作业 2：CartoPy 综合制图演练

基于 `cartopy` 绘制两个年份上述湖泊面积的对比图。要求使用原始获取的遥感图像作为底图，并在上面叠加提取的湖泊面积矢量图层。